

Organização e Recuperação de Dados

2º Trabalho Prático

A especificação abaixo deverá ser implementada utilizando a linguagem Python e deverá ser realizada em equipes de 3 alunos. A equipe deverá enviar o código fonte pelo *Google Classroom* (apenas arquivos .py). Posteriormente, a equipe **deverá apresentar o trabalho em data e horário agendado pela professora. A entrega do código é um pré-requisito para a avaliação, mas a nota dependerá do desempenho de cada integrante da equipe na apresentação. O uso deliberado de IA generativa acarretará diminuição na nota.**

Especificação

O objetivo deste trabalho é criar um programa que, a partir de um arquivo de registros, construa e mantenha **um índice estruturado como uma árvore-B**.

O arquivo de registros conterá informações sobre jogos e estará armazenado no arquivo **games.dat** no mesmo formato utilizado no 1º trabalho prático. Para cada registro lido, a chave primária (o identificador do jogo) deverá ser inserida em uma árvore-B, juntamente com o *byte-offset* do registro correspondente. Para facilitar a experimentação com árvores de diferentes ordens, **defina ORDEM como uma constante global** e use-a ao longo do código sempre que precisar referenciar a ordem da árvore. **Esteja ciente de que o seu programa será testado com árvores de diferentes ordens.**

A estrutura interna das páginas da árvore-B será similar à vista em aula, porém deverá conter, adicionalmente, espaço para o armazenamento dos *byte-offsets* dos registros. **A árvore-B deverá ser criada e mantida em arquivo, logo, nunca estará completamente carregada na memória.**

O programa deverá oferecer as seguintes funcionalidades:

- Criação do índice (árvore-B) a partir do arquivo de registros (opção -b);
- Execução de um arquivo de operações (apenas busca e inserção) (opção -e);
- Impressão das informações do índice, i.e., da árvore-B (opção -p).

Criação do índice (i.e., a árvore-B)

A funcionalidade de criação do índice será acessada pela linha de comando, da seguinte forma:

```
$ python programa.py -b
```

sendo `programa.py` o nome do arquivo com o seu código e `-b` a flag que sinaliza o modo de criação do índice. Sempre que ativada, essa funcionalidade lerá o arquivo **games.dat** e fará a inserção dos pares {chave, *byte-offset*} na árvore-B que deverá ser armazenada em um **arquivo binário chamado btree.dat**. Caso o arquivo `btree.dat` exista, ele deverá ser sobrescrito. Note que o formato do arquivo `games.dat` será sempre o mesmo, porém **o número de registros nesse arquivo pode variar**. Para simplificar o processamento do arquivo de registros, considere que ele sempre será fornecido corretamente (i.e., o seu programa não precisa verificar a integridade desse arquivo). **Neste trabalho, não serão feitas remoções** e, conseqüentemente, não haverá gerenciamento de espaços removidos.

Ao final da criação do índice, o programa deverá apresentar uma mensagem na tela indicando se a operação foi concluída com sucesso ou não.

Execução do arquivo de operações

Dados os arquivos **games.dat** e **btree.dat**, o seu programa deverá executar as seguintes operações:

- Busca de um jogo pelo ID;
- Inserção de um novo jogo.

As operações a serem realizadas em determinada execução serão especificadas em um **arquivo de operações**. Dessa forma, **o programa não possuirá interface com o usuário** e executará as operações na sequência em que estiverem especificadas no arquivo de operações.

A execução do arquivo de operações será acionada pela linha de comando, no seguinte formato:

```
$ python programa.py -e nome_arquivo_operacoes
```

sendo `programa.py` o nome do arquivo com o seu código, `-e` a flag que sinaliza o modo de execução e `nome_arquivo_operacoes` o nome do arquivo que contém as operações a serem executadas. Para simplificar o processamento do arquivo de operações, considere que ele sempre será fornecido corretamente (i.e., o seu programa não precisa verificar a integridade desse arquivo).

Observe que, para esse tipo de execução, os arquivos **games.dat** e **btree.dat** devem existir. Caso eles não existam, o programa deve apresentar uma mensagem de erro e terminar.

Formato do arquivo de operações

O arquivo de operações possuirá uma operação por linha, codificada com o identificador da operação seguido pelo seu argumento. Os identificadores de operações possíveis são: `b` = *busca* e `i` = *inserção*. Considere como exemplo o arquivo de operações abaixo:

```
b 220
i 14|The Last of Us|2013|Action-Adventure|Sony|PlayStation 3|
b 9
b 238
i 1001|Pac-Man|1980|Maze|Namco|Arcade|
i 14|The Last of Us|2013|Action-Adventure|Sony|PlayStation 3|
```

Esse arquivo representa a execução consecutiva das seguintes operações:

- Busca pelo registro de chave 220
- Inserção do registro do jogo de identificador 14 (“The Last of Us”)
- Busca pelo registro de chave 9
- Busca pelo registro de chave 238
- Inserção do registro do jogo de identificador 1001 (“Pac-Man”)
- Inserção do registro do jogo de identificador 14 (“The Last of Us”)

As buscas deverão ser realizadas no índice que está armazenado no arquivo `btree.dat`. Uma vez localizada a chave no índice, o *byte-offset* do registro correspondente deverá ser recuperado e o registro deverá ser acessado de forma direta no arquivo `games.dat`.

As inserções sempre acontecerão no fim do arquivo `games.dat` e, complementarmente, as informações do novo registro (chave e *byte-offset*) deverão ser inseridas no índice do arquivo **`btree.dat`**. Não será permitida a inserção de registros com chave duplicada. Dessa forma, **antes de realizar uma inserção, o índice deverá ser consultado** e uma mensagem de erro deverá ser dada caso se identifique a duplicação.

Utilizando como exemplo o arquivo de operações mostrado acima, o seu programa deverá apresentar:

```
Busca pelo registro de chave "220"
220|Assassin's Creed 2|2009|Action-Adventure|Ubisoft|PlayStation 3| (67 bytes - offset 4244)

Inserção do registro de chave "14"
14|The Last of Us|2013|Action-Adventure|Sony|PlayStation 3| (59 bytes - offset 9416)

Busca pelo registro de chave "9"
Erro: chave "9" não encontrada

Busca pelo registro de chave "238"
238|NieR: Automata|2017|Hack and Slash|Square Enix|PlayStation 4| (65 bytes - offset 3337)

Inserção do registro de chave "1001"
1001|Pac-Man|1980|Maze|Namco|Arcade| (36 bytes - offset 9477)

Inserção do registro de chave "14"
Erro: chave "14" duplicada

As operações do arquivo "operacoes.txt" foram executadas com sucesso!
```

Impressão das informações da árvore-B

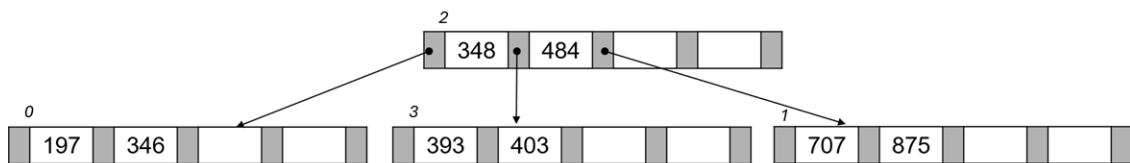
A funcionalidade de impressão das informações da árvore-B (-p) também será acessada via linha de comando, no seguinte formato:

```
$ python programa.py -p
```

sendo `programa.py` o nome do arquivo com o seu código. Note que, para esse tipo de execução, o arquivo **btree.dat** deve existir. Caso o arquivo contrário, o programa deve apresentar uma mensagem de erro e terminar.

Sempre que ativada, essa funcionalidade apresentará na tela o conteúdo de todas as páginas da árvore-B armazenada no arquivo **btree.dat**. Para cada página da árvore deverá ser informado: (a) o seu RRN; (b) os valores das chaves; (c) os valores dos *byte-offsets* dos registros associados às chaves; e (d) os RRNs das páginas filhas. **As páginas devem ser apresentadas pela ordem do seu RRN e a página raiz deve ser devidamente identificada.**

Como um exemplo, considere uma árvore-B de ordem 5 que indexa um arquivo com os oito primeiros registros do arquivo `games.dat`.



Supondo a árvore-b estruturada acima, seu programa deverá apresentar as seguintes informações:

```
Página 0:
Chaves = 197 | 346 | -1 | -1
Offsets = 342 | 456 | -1 | -1
Filhos = -1 | -1 | -1 | -1 | -1

Página 1:
Chaves = 707 | 875 | -1 | -1
Offsets = 136, 275, -1 | -1
Filhos = -1 | -1 | -1 | -1 | -1

- - - - - Raiz - - - - -
Página 2:
Chaves = 348 | 484 | -1 | -1
Offsets = 93 | 0 | -1 | -1
Filhos = 0 | 3 | 1 | -1 | -1

- - - - -
Página 3:
Chaves = 393, 403, -1 | -1
Offsets = 189, 392, -1 | -1
Filhos = -1 | -1 | -1 | -1 | -1
```

Qualquer dúvida referente a esta especificação deverá ser sanada com os respectivos professores.

BOM TRABALHO!